[3 pts] What is the purpose of system calls?

- System calls are the interface between the user space and the kernel space. The main purpose of the system calls is to allow the user space to request services and resources to access the hardware resources. This can include the CPU, memory disk and more. In following this process, it also ensures that these resources accessed by the system calls are accessed in a secure and stable manner as only the kernel space can access them. Therefore, it also prevents buggy programs from disturbing the entire OS. Also, system calls create a level of abstraction between the hardware operation and the user space. Therefore, the user programs will be able to complete their necessary tasks without needing to interact with a computers' hardware directly. This ultimately makes user programs more efficient as they are now portable across multiple different systems with the same operating system as well.

[3 pts] How does a system call execute? Explain the steps from making the call in the user space process to returning from the call with a result.

- Once the user space invokes a system call, it starts the entire process. It begins by using a wrapper function to set up the parameters provided by the user space and retrieves the corresponding system call number. The program will then execute the needed instruction, like syscall, which alerts it to switch into kernel mode. Once here, the kernel mode is able to identify which specific system call is needed, grabs all the parameters provided by the user mode, and forces the kernel mode to perform the operation required, like reading or opening a file. Once the kernel mode completes this task, it places the results back into a specific register and returns back to the user mode. The user mode will retrieve the results created and continue onto the next task at hand.

[4 pts] What is the purpose of interrupts? How does an interrupt differ from a trap? Can traps be generated intentionally by a user program? If so, for what purpose?

- The purpose of an interrupt is to create a fallback mechanism which allows the OS to regain control. In essence, it stops the current execution that is occurring and forces the OS to move into the next specific task. Interrupts are different from a trap because interrupts are unexpected and asynchronous. An example of this would be an unexpected click of a key on a keyboard from a user that requires the CPU attention immediately. On the other hand, a trap is a deliberate and synchronous action, such as a system call from the user program. An example of this would be if a program raises a trap when a specific exception happens within code due to invalid logic or syntax and sets a debugging error to the user. With this specific example, it can be understood that traps can be generated intentionally by a user program for further debugging purposes.